

```

      1 |      2 |      3
      3 |      1 |      3
(3 rows)

```

Or if we want to remove all duplicate rows, we will have to run the following:

```

forumbd=> select distinct category, count(*) over (partition by
category),count(*) over ()
from posts
order by category;
  category | count | count
-----+-----+-----
         1 |      2 |      3
         3 |      1 |      3
(2 rows)

```

In the preceding query, the first window function aggregates the data using the category field, while the second one aggregates the data of the whole table.

Using the window functions, it is possible to aggregate the data in different fields in the same query.

As we've seen here, we can define the window frame directly at the query level, but we can also define an alias for the window frame. For example, the preceding query becomes the following:

```

forumbd=> select distinct category, count(*) over w1 ,count(*) over W2
from posts
WINDOW w1 as (partition by category),W2 as ()
order by category;
  category | count | count
-----+-----+-----
         1 |      2 |      3
         3 |      1 |      3
(2 rows)

```

The use of aliases is called the WINDOW clause. The WINDOW clause is very useful when we have many aggregates.

Introducing some useful functions

Window functions can use all the aggregation functions that we explored in the previous chapter. In addition to these, window functions introduce new aggregation functions.

Before we examine some of those, let's introduce a unique function – `generate_series`. `generate_series` simply generates a numerical series, for example:

```
forumbd=> select * from generate_series(1,5);
generate_series
-----
          1
          2
          3
          4
          5
(5 rows)
```

In the following examples, we will use this function for various use cases.

The ROW_NUMBER function

Now let's look at the `ROW_NUMBER()` function. The `ROW_NUMBER()` function assigns a progressive number for each row within the partition:

```
forumbd=> select category, row_number() over w from posts WINDOW w as
(partition by category) order by category;
category | row_number
-----+-----
          1 |          1
          1 |          2
          3 |          1
(3 rows)
```

In the preceding query, we've used the `PARTITION BY` clause to divide the window into subsets based on the values in the `category` column. As can be seen, we have two category values: 1 and 3. This means that we have two windows and inside each window, the `ROW_NUMBER()` function assigns numbers as we defined before.

The ORDER BY clause

The `ORDER BY` clause sorts the values inside the window. We can also use the `NULLS FIRST` or `NULLS LAST` option to have the null values at the beginning or at the end of the sorting. For example, we can perform a window function query without an `ORDER BY` clause, as we can see in the following snippet, but we have to pay attention to what kind of function we are using, and what our goal is.

If we use aggregation functions that do not depend on the sort order, such as the COUNT function, we can avoid sorting the data; otherwise, it is good practice to sort the data inside the partition in order to avoid the risk of having different results every time the query is launched:

```

forumbd=> select category,row_number() over w,title
from posts WINDOW w as (partition by category order by title) order by
category;
category | row_number | title
-----+-----+-----
          | 1 | Indexing Mysql
          | 2 | Indexing PostgreSQL
          | 3 | A view of Data types in C++
(3 rows)

```

As we can see, inside the partition, the data is sorted on the title field.

FIRST_VALUE

The FIRST_VALUE function returns the first value within the partition, for example:

```

forumbd=> \x
Expanded display is on.

forumbd=> select category,row_number() over w,title,first_value(title)
over w
from posts WINDOW w as (partition by category order by category) order by
category;
-[ RECORD 1 ]-----
category    | 1
row_number  | 1
title       | Indexing PostgreSQL
first_value | Indexing PostgreSQL
-[ RECORD 2 ]-----
category    | 1
row_number  | 2
title       | Indexing Mysql
first_value | Indexing PostgreSQL
-[ RECORD 3 ]-----
category    | 3
row_number  | 1

```

```

title      | A view of  Data types in C++
first_value | A view of  Data types in C++

```

LAST_VALUE

The LAST_VALUE function returns the last value within the partition, for example:

```

forumdb=> select category,row_number() over w,title,last_value(title) over
w
from posts WINDOW w as (partition by category order by category) order by
category;
-[ RECORD 1 ]-----
category | 1
row_number | 1
title     | Indexing PostgreSQL
last_value | Indexing Mysql
-[ RECORD 2 ]-----
category | 1
row_number | 2
title     | Indexing Mysql
last_value | Indexing Mysql
-[ RECORD 3 ]-----
category | 3
row_number | 1
title     | A view of  Data types in C++
last_value | A view of  Data types in C++

```

It is important to always use the Order by clause when we use the first_value() or last_value() function to avoid incorrect results, as mentioned previously.

RANK

The RANK function ranks the current row within its partition with gaps. If we don't specify a PARTITION BY clause, the function doesn't know how to correlate the current tuple, so the function correlates to itself, as seen here:

```

forumdb=> select pk,title,author,rank() over () from posts ;
pk | title | author | rank
----+-----+-----+-----
5 | Indexing PostgreSQL | 1 | 1
6 | Indexing Mysql | 1 | 1

```

```

 7 | A view of  Data types in C++ |      2 |      1
(3 rows)

```

If we add the `order by` clause, the function ranks in the assigned order, for example, the author with `id 1` starts from record 1, and the author with `id 2` starts from record 3, as we can see in the following example:

```

forumdb=> select pk,title,author,rank() over (order by author) from posts
;
pk |          title          | author | rank
----+-----+-----+-----+-----
 5 | Indexing PostgreSQL     |      1 |    1
 6 | Indexing Mysql          |      1 |    1
 7 | A view of  Data types in C++ |      2 |    3
(3 rows)

```

If we add the `PARTITION BY` clause, the working mechanism is the same; the only difference is that the ranking is calculated within the partition and not on the whole table as in the previous example:

```

forumdb=> select pk,title,author,rank() over (partition by author order by
author) from posts ;

pk |          title          | author | rank
----+-----+-----+-----+-----
 5 | Indexing PostgreSQL     |      1 |    1
 6 | Indexing Mysql          |      1 |    1
 7 | A view of  Data types in C++ |      2 |    1
(3 rows)

```

DENSE_RANK

The `DENSE_RANK` function is similar to the `RANK` function. The difference is that the `DENSE_RANK` function ranks the current row within its partition without gaps:

```

forumdb=> select pk,title,author,dense_rank() over (order by author) from
posts order by category;
pk |          title          | author | dense_rank
----+-----+-----+-----+-----
 5 | Indexing PostgreSQL     |      1 |          1
 6 | Indexing Mysql          |      1 |          1

```

```
7 | A view of Data types in C++ | 2 | 2
(3 rows)
```

The LAG and LEAD functions

In this section, we will show how the LAG and LEAD functions work. First of all, we are going to set up our environment and generate a sequence of numbers as we did previously:

```
forumdb=> select x from generate_series(1,5) as x;
 x
---
 1
 2
 3
 4
 5
(5 rows)
```

This is our starting point for this example. The official documentation (<https://www.postgresql.org/docs/current/functions-window.html>) defines the LAG function as follows:



The LAG function returns a value evaluated at the row that is offset rows before the current row within the partition; if there is no such row, it instead returns the default (which must be of the same type as the value). Both the offset and the default are evaluated with respect to the current row. If omitted, offset defaults to 1 and default to null.

Now, let's write the following statement:

```
forumdb=> select x,lag(x) over w from (select generate_series(1,5) as x) V
WINDOW w as (order by x) ;
 x | lag
---+-----
 1 |
 2 |  1
 3 |  2
 4 |  3
 5 |  4
(5 rows)
```

As we can see, the `lag` function returns a result set with an offset value equal to 1. If we introduce an offset parameter, the `lag` function will return a result set with an offset equal to the number that we have passed as input, as can be seen in the next example:

```
forumbd=> select x,lag(x,2) over w from (select generate_series(1,5) as x)
V WINDOW w as (order by x) ;
 x | lag
---+-----
 1 |
 2 |
 3 |  1
 4 |  2
 5 |  3
(5 rows)
```

The `lead` function is the opposite of the `lag` function, as described in the official documentation:

“The `LEAD` function returns the value evaluated at the row that is offset rows after the current row within the partition; if there is no such row, it instead returns the default (which must be of the same type as the mentioned value). Both the offset and default are evaluated with respect to the current row. If omitted, the offset defaults to 1 and the default becomes null.”

Here are a couple of examples where we can see how it works. In the first example, we will use the `lead` function without any parameters:

```
forumbd=# select x,lead(x) over w from (select generate_series(1,5) as x)
V WINDOW w as (order by x) ;
 x | lead
---+-----
 1 | 2
 2 | 3
 3 | 4
 4 | 5
 5 |
(5 rows)
```

As we can see in the `lead` function, the offset starts from the bottom.

Let's now see an example of using the lead function with an offset parameter:

```
forumbd=> select x,lead(x,2) over w from (select generate_series(1,5) as
x) V WINDOW w as (order by x) ;
 x | lead
---+-----
 1 |      3
 2 |      4
 3 |      5
 4 |
 5 |
(5 rows)
```

The CUME_DIST function

The CUME_DIST function calculates the cumulative distribution of a value within a partition. The function is described in the official documentation as follows:

“The CUME_DIST function computes the fraction of partition rows that are less than or equal to the current row and its peers.”

Let's look at an example:

```
forumbd=> select x,cume_dist() over w from (select generate_series(1,5) as
x) V WINDOW w as (order by x) ;
 x | cume_dist
---+-----
 1 |         0.2
 2 |         0.4
 3 |         0.6
 4 |         0.8
 5 |         1
(5 rows)
```

As the function is mathematically defined, the cume_dist function can never have a value greater than the current value of the field.

The NTILE function

The PostgreSQL NTILE function groups the rows sorted in the partition. Starting from 1, up to the parameter value passed to the NTILE function, each group is assigned a number of buckets.

The parameter passed to the NTILE function determines how many records we want the bucket to be composed of.

Now, let's see an example of how it works by trying to split our result set into two buckets:

```
forumbd=> select x,ntile(2) over w from (select generate_series(1,6) as x)
V WINDOW w as (order by x) ;
 x | ntile
---+-----
 1 |      1
 2 |      1
 3 |      1
 4 |      2
 5 |      2
 6 |      2
(6 rows)
```

If we wanted to divide our result set into three buckets, we would run the following statement:

```
forumbd=> select x,ntile(3) over w from (select generate_series(1,6) as x)
V WINDOW w as (order by x) ;
 x | ntile
---+-----
 1 |      1
 2 |      1
 3 |      2
 4 |      2
 5 |      3
 6 |      3
(6 rows)
```

The NTILE() function accepts an integer and tries to divide the window into a number of balanced buckets, specifying to which bucket each row belongs.

In this section, we have introduced some features that allow you to do some basic data mining. For example, lag and lead could be used to compare different lines of a table, and therefore compare the salaries of different employees, or compare collections from different days.

In the next section, we will go into even more detail and explore some more advanced features of window functions.

Using advanced statement window functions

In this section, we will discuss advanced window functions in detail, and we will explore some techniques that may be useful for carrying out more detailed data analysis.

Let's start with a query that we saw at the start of this chapter:

```
forumbd=> select distinct category, count(*) over (partition by category)
from posts order by category;
  category | count
-----+-----
          1 |      2
          3 |      1
(2 rows)
```

Here, below, there is another way to write the same aggregate that we described before:

```
forumbd=> select distinct category, count(*) over w1
from posts WINDOW w1 as (partition by category RANGE BETWEEN UNBOUNDED
PRECEDING AND CURRENT ROW)
order by category;
  category | count
-----+-----
          1 |      2
          3 |      1
(2 rows)
```

What does `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` mean? They are the default conditions, known as the **frame clause**. This means that the data is partitioned, first by category, and then within the partition, and the count is calculated by resetting the count every time the frame is changed.

The frame clause

In this section, we'll talk about the frame clause, which allows us to manage partitions in a different way. The frame clause has two forms:

- Rows between `start_point` and `end_point`
- Range between `start_point` and `end_point`